MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Prompt engineering tool for LLMs

Master's Thesis

BC. MAXIM SVISTUNOV

Brno, Fall 2025

MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Prompt engineering tool for LLMs

Master's Thesis

BC. MAXIM SVISTUNOV

Advisor: Ing. Aleš Studený

Department of Computer Systems and Communications

Brno, Fall 2025

MUNI
FI

Declaration

Hereby I declare that this thesis is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, I used the following AI tools:

- Claude for assistance with developing the application and preparing the thesis document
- Gemini for assistance with minor tasks

I declare that I used these tools in accordance with the principles of academic integrity. I checked the content and took full responsibility for it.

Bc. Maxim Svistunov

Advisor: Ing. Aleš Studený

Acknowledgements

I would like to thank my advisor Ing. Aleš Studený for his guidance throughout this work.

Abstract

This thesis presents the design and implementation of PromptForge, a prompt engineering tool for large language models implemented as a Chrome browser extension. The tool introduces instruction-based prompt composition, enabling users to build prompts from reusable text fragments stored in a personal library. Key features include highly optimized library management and prompt engineering workflow, as well as AI-assisted prompt refinement and instruction extraction from conversations.

Keywords

LLM, AI tools, prompt engineering, instruction composition

Contents

List of Figures

1 Introduction

1.1 Problem Statement

The emergence of large language models (LLMs) has fundamentally transformed how humans interact with artificial intelligence systems. Services such as ChatGPT, Claude, and Gemini have made sophisticated AI capabilities accessible to millions of users through conversational web interfaces. However, the quality of outputs from these systems depends critically on how users formulate their requests—a practice now widely known as prompt engineering.

1.1.1 Manual prompt engineering is tedious and error-prone

Effective prompting often requires including specific instructions that guide the AI's behavior: requests to explain reasoning, to consider multiple perspectives, to format output in particular ways, or to ask clarifying questions before proceeding. Users who discover effective prompting patterns find themselves repeatedly typing similar instructions across different conversations. This manual repetition is not only tedious but also error-prone—slight variations in wording can lead to inconsistent results, and valuable prompting strategies are easily forgotten or lost.

1.1.2 Prompts are non-reusable across conversations

Each conversation with a webchat AI begins with a blank slate. There is no built-in mechanism to carry forward successful prompting strategies from previous interactions. Users who develop sophisticated approaches to particular types of tasks—code review, document analysis, creative writing—must reconstruct these approaches from memory each time. This limitation represents a significant barrier to building expertise, as the knowledge encoded in effective prompts remains ephemeral.

1.1.3 Lack of systematic prompt organization

Beyond individual prompts, users lack tools for systematically organizing and discovering prompting strategies. A user might recall that a particular approach worked well for a specific task but cannot easily retrieve the exact wording. There is no searchable library, no categorization system, and no way to build upon the accumulated knowledge of what works. This absence of tooling stands in contrast to other domains of knowledge work, where filing systems, search engines, and organizational tools are considered essential.

1.2 PromptForge as Solution

PromptForge addresses these challenges by providing a prompt engineering environment that treats reusable text fragments—called /instructions/—as first-class entities. Rather than typing prompts from scratch each time, users compose them by selecting and combining instructions from a personal library. The tool operates as a Chrome browser extension, integrating directly with popular AI chat interfaces to minimize context switching and friction.

The fundamental insight behind PromptForge is that prompt engineering benefits from the same principles that have proven valuable in software development: modularity, reusability, and systematic organization. Just as programmers build complex systems from well-defined functions and libraries, prompt engineers can construct sophisticated prompts from well-defined instructions. This approach transforms prompt engineering from an ad-hoc practice into a systematic discipline.

1.3 Core Principles

Five principles guided the design of PromptForge:

1.3.1 Instruction-based composition

The central abstraction in PromptForge is the *instruction*: a reusable text snippet designed to achieve a specific effect when included in a

prompt. Instructions are the building blocks from which prompts are composed. They can be combined, nested within each other, and parameterized with variables, enabling users to build complex prompts from simple, well-understood components.

1.3.2 Low friction

Inserting instructions into prompts must be effortless. If the overhead of using the tool exceeds the effort of typing, users will abandon it. PromptForge prioritizes keyboard-driven interactions, offering a command palette (Quick Inclusion) that allows users to search and insert instructions without leaving the keyboard. Double-clicking, drag-and-drop, and explicit mode selection provide alternative interaction paths for different preferences.

1.3.3 Discoverability

Users must be able to find relevant instructions whether they know exactly what they are looking for (search) or wish to explore available options (browse). PromptForge provides both: fuzzy search across instruction names, descriptions, and content; hierarchical folder organization; tag-based filtering; and a tree view that supports both navigation and direct manipulation.

1.3.4 AI-assisted refinement

The tool leverages the very AI systems it helps users interact with. Rather than requiring a separate AI backend, PromptForge injects carefully crafted meta-prompts into the user's webchat conversation to request improvements (Refine workflow) or extract new instructions from successful interactions (Harvest workflow). This design eliminates API key management and ensures that users work with the AI model they have already chosen.

1.3.5 Browser integration

PromptForge operates where users work: inside their browser, alongside the AI chat interface. The extension injects a sidebar that coexists

with ChatGPT, Claude, or Gemini, enabling direct content transfer between the tool and the webchat. This integration eliminates copy-paste friction and enables workflows that would be impossible with a standalone application.

1.4 Thesis Goals and Contributions

This thesis presents the design and implementation of PromptForge with the following goals:

1. **Design a data model for reusable prompt instructions** that supports composition, parameterization, and cross-referencing while remaining simple enough for users to understand and extend.

2. **Implement a browser extension architecture** that integrates seamlessly with multiple AI chat interfaces without requiring server-side components or API credentials.

3. **Develop AI-assisted workflows** (Refine and Harvest) that leverage the webchat AI to help users improve prompts and grow their instruction libraries.

4. **Create a keyboard-driven user interface** that minimizes friction while supporting both novice exploration and expert efficiency.

5. **Document the design decisions and trade-offs** encountered during development, providing insights for future tool builders in this emerging space.

The primary contribution of this work is a functioning tool that demonstrates how prompt engineering can be transformed from an ad-hoc practice into a systematic discipline through appropriate tooling. Secondary contributions include the specific techniques developed for AI-assisted prompt refinement and instruction extraction.

1.5 Document Structure

The remainder of this thesis is organized as follows:

Chapter 2: Background establishes terminology, reviews relevant literature on prompt engineering and human-AI collaboration, and positions PromptForge relative to existing tools.

Chapter 3: Architecture describes the technical architecture of the Chrome extension, including the technology stack, storage strategy, and webchat integration approach.

Chapter 4: Data Model details the instruction format, variable system, embedding mechanism, and prompt storage.

Chapter 5: User Interface presents the interface layout with emphasis on the Quick Inclusion feature that enables rapid instruction search and insertion.

Chapter 6: Core Workflows explains the primary user workflows: webchat integration (Inject/Pull), insertion modes (Insert/Wrap/Prepend/Append), the Refine workflow for AI-assisted improvement, and the Harvest workflow for instruction extraction.

Chapter 7: AI Prompt Engineering analyzes the design of the built-in AI instruction templates (Refine, Harvest, Tutorial), examining the prompt engineering techniques employed.

Chapter 8: Development Journey recounts the evolution of the tool through three architectural phases and discusses lessons learned.

Chapter 9: Future Work identifies directions for further development.

Chapter 10: Conclusion summarizes contributions and reflects on the achievement of thesis goals.

Appendices provide keyboard shortcut reference and complete AI instruction templates.

2 Background

This chapter establishes the conceptual and terminological foundation for the thesis. We begin by defining key terms, then review relevant literature on prompt engineering and human-AI collaboration, and finally position PromptForge relative to existing tools in this space.

2.1 Terminology

Precise terminology is essential for discussing prompt engineering tools. This section provides formal definitions for the key concepts used throughout this thesis.

2.1.1 Prompt and prompt engineering

A **prompt** is the textual input provided to a large language model to elicit a response. In conversational AI interfaces, prompts take the form of user messages in a chat dialogue. The term encompasses both simple queries ("What is the capital of France?") and complex, structured requests that include context, constraints, examples, and specific instructions for how the AI should behave.

Prompt engineering refers to the practice of crafting prompts to achieve desired outputs from language models. Schulhoff et al. [1] define prompt engineering as "the process of structuring text that can be interpreted and understood by a generative AI model." The field has rapidly evolved from ad-hoc experimentation to a more systematic discipline with documented techniques, taxonomies, and best practices.

The significance of prompt engineering stems from the sensitivity of language models to input phrasing. Small changes in wording, structure, or included context can produce substantially different outputs. This sensitivity makes prompt engineering both powerful—skilled practitioners can elicit sophisticated behaviors—and challenging—effective patterns must be discovered, documented, and consistently applied.

2.1.2 Instruction

In the context of PromptForge, an **instruction** is a reusable text fragment designed to achieve a specific effect when included in a prompt. Instructions are the atomic units of composition in the tool's data model.

The term is chosen deliberately to distinguish from "prompt" (which refers to the complete input) and to emphasize the directive nature of these fragments. Instructions typically guide AI behavior rather than pose questions: "Explain your reasoning step by step," "Consider both advantages and disadvantages," "Ask clarifying questions before proceeding."

Instructions differ from prompt templates (discussed below) in that they are designed to be combined freely rather than filled in. A prompt template provides a fixed structure with placeholders; an instruction provides a discrete capability that can be mixed with other instructions as needed.

2.1.3 AI and LLM

Artificial Intelligence (AI) in this thesis refers specifically to generative AI systems accessed through conversational interfaces, commonly called "AI chatbots" or "AI assistants." We use "AI" as the user-facing term that matches how these systems are marketed and discussed.

Large Language Model (LLM) refers to the underlying technology: neural network models trained on large text corpora that generate text by predicting likely continuations. Contemporary LLMs such as GPT-4, Claude, and Gemini power the conversational AI services with which PromptForge integrates. While the technical distinction between the model and the service is sometimes relevant, we use "AI" and "LLM" largely interchangeably when the distinction is not material.

Webchat AI refers specifically to browser-based conversational interfaces to LLMs—services like ChatGPT (chat.openai.com), Claude (claude.ai), and Gemini (gemini.google.com). These are the integration targets for PromptForge.

2.2 Prompt Engineering Literature Review

The academic study of prompt engineering has grown rapidly alongside the deployment of large language models. This section reviews key literature relevant to PromptForge's design.

2.2.1 Survey papers and technique taxonomies

Several comprehensive surveys have attempted to systematize knowledge about prompt engineering. The most extensive is "The Prompt Report" by Schulhoff et al. [1], which conducted a PRISMA systematic review of 1,565 papers. This survey establishes 33 vocabulary terms and catalogs 58 text-only prompting techniques plus 40 additional techniques for multimodal models. The taxonomy organizes techniques into categories including zero-shot prompting, few-shot prompting, thought generation (chain-of-thought and variants), decomposition, ensembling, and self-criticism.

Sahoo et al. [2] provide a complementary categorization of 29 techniques organized by their functional role: reasoning enhancement, knowledge augmentation, output formatting, and constraint enforcement. Their work emphasizes how different techniques address different classes of prompting challenges.

Chen et al. [3] survey advanced techniques including chain-of-thought prompting, self-consistency, and generated knowledge approaches, with particular attention to security considerations and adversarial attacks on prompts.

These surveys establish that prompt engineering is not merely about phrasing requests clearly, but involves a rich set of techniques for structuring interactions with language models. PromptForge's instruction-based approach provides a mechanism for encapsulating and reusing these techniques.

2.2.2 Reusable prompts and templates in literature

The concept of reusable prompt components has emerged in both academic research and practitioner communities. White et al. [4] propose a "prompt pattern catalog" that draws explicit parallels to software design patterns. Their catalog identifies recurring prompt structures that

solve common problems: the Persona Pattern (assigning a role to the AI), the Template Pattern (providing output structure), the Cognitive Verifier Pattern (asking the AI to generate clarifying questions), and others. This pattern-based thinking directly informs PromptForge's instruction concept.

The Prompt Canvas framework [5] provides a practitioner-oriented guide that consolidates techniques into a unified framework. Significantly, it discusses "pre-designed templates" and "reusable prompts" as mechanisms for operationalizing prompt engineering knowledge, validating the need for tooling support.

Wordflow [6] describes an academic prototype combining a "Personal Prompt Library" with a "Community Prompt Hub." Their template system uses prefix text and placeholder variables—an approach similar to PromptForge's variable syntax. Wordflow's research demonstrates user demand for prompt reuse and sharing mechanisms.

The industrial analysis "From Prompts to Templates" [7] examines prompt template usage in production LLM applications at Uber and Microsoft. This work provides empirical evidence that template-based approaches scale to enterprise use and categorizes the types of placeholders commonly used in production templates.

2.2.3 Human-AI collaboration research

PromptForge is fundamentally a human-AI collaboration tool, and its design draws on HCI research in this area. The systematic review "Co-Writing with AI" [8] analyzes 109 HCI papers on AI writing assistance, identifying four design strategies: structured guidance, guided exploration, active co-writing, and critical feedback. PromptForge primarily embodies the "structured guidance" strategy, providing users with predefined instructions that they select and combine.

Research on scaffolding in human-AI collaboration [9] examines how varying levels of AI input intensity affect user experience and writing outcomes. This work informs PromptForge's design philosophy of user control: the tool facilitates instruction discovery and insertion but leaves composition decisions to the user.

Empirical studies of human-AI writing collaboration [10] have documented the patterns that emerge when users work with AI writing tools, finding that successful collaboration often involves iterative

refinement and explicit behavioral guidance—precisely the use case PromptForge addresses.

2.3 Related Work

2.3.1 Existing prompt management tools

Several tools address aspects of prompt management, though none fully matches PromptForge's scope and approach.

Prompt libraries and marketplaces: Services like PromptBase and various GitHub repositories offer collections of prompts for specific use cases. These are static collections without tooling support for organization, search, or integration with webchat interfaces.

Browser extensions for prompt insertion: Extensions like AIPRM provide template libraries integrated with ChatGPT. These typically offer predefined prompts selected from a catalog rather than composable instructions, and often focus on SEO and marketing use cases rather than general prompt engineering.

Clipboard managers and text expanders: Tools like TextExpander or Alfred snippets can store and insert text fragments. However, they lack awareness of prompt engineering concepts—no variable substitution, no instruction composition, no AI-assisted refinement.

IDE integrations: Tools like GitHub Copilot and Cursor integrate AI assistance into code editors. While sophisticated, these target software development rather than general prompt engineering and do not support the webchat interfaces where many users interact with AI.

API-based prompt management: Platforms like LangChain and LlamaIndex provide programmatic prompt management for developers building AI applications. These target a different user population (developers integrating LLMs into applications) rather than end users composing prompts in webchat interfaces.

2.3.2 Positioning of PromptForge

PromptForge occupies a specific position in this landscape: a tool for end users (not developers) who interact with AI through webchat

interfaces (not APIs) and who want to build personal libraries of reusable instructions (not just select from predefined templates).

Key differentiators include:

1. **Instruction composition:** Unlike template-based tools that offer complete prompts, PromptForge treats instructions as composable units that can be freely combined.

2. **Browser integration:** The extension injects directly into webchat interfaces, enabling seamless content transfer without copy-paste.

3. **AI-assisted workflows:** The Refine and Harvest features leverage the webchat AI itself, requiring no separate API configuration.

4. **Local-first storage:** User data resides in a user-selected folder on their filesystem, ensuring privacy and enabling backup/sync through existing tools.

5. **Keyboard-driven efficiency:** Quick Inclusion provides FZF-style fuzzy search for rapid instruction insertion without leaving the keyboard.

The closest prior work is Wordflow [6], which shares the concept of a personal prompt library with template variables. PromptForge extends this concept with composition (instructions can embed other instructions), AI-assisted refinement (the Refine workflow), instruction extraction (the Harvest workflow), and deep browser integration (sidebar injection rather than a separate application).

3 Architecture

This chapter describes the technical architecture of PromptForge, explaining the design decisions that shaped the implementation. We begin with a high-level overview, then detail the technology stack, Chrome extension structure, and storage architecture. The chapter concludes with a discussion of how the architecture evolved through multiple iterations.

3.1 Overview

3.1.1 Pure Chrome extension approach

PromptForge operates as a pure Chrome browser extension with no server-side components. This architectural decision has profound implications for deployment, privacy, and capability.

The extension consists of three primary components: a background service worker that handles storage and message routing, content scripts that inject the sidebar UI into supported webchat pages, and the React-based sidebar application itself.

3.1.2 High-level component diagram

The component structure follows Chrome's Manifest V3 extension model [11]:

Background Service Worker (TypeScript) Handles storage coordination and message routing between components.

Content Script (TypeScript) Manages sidebar injection and provides access to the webchat page's DOM.

Sidebar UI (React 18) The main user interface where users compose prompts and manage their library.

Options Page (React 18) Settings interface for configuring extension behavior.

Communication between components uses Chrome's message passing APIs. The content script acts as a bridge between the isolated sidebar (running in an injected iframe) and the webchat page's DOM.

3.1.3 Rationale for browser-based approach

The decision to implement PromptForge as a pure browser extension rather than a standalone application or web service was driven by several factors:

Integration fidelity: A browser extension can inject UI directly into webchat pages, enabling seamless interaction with the chat interface. Standalone applications would require copy-paste or complex system-level integration.

Zero-configuration deployment: Users install from the Chrome Web Store with one click. No server setup, no API keys for basic functionality, no account creation required.

Privacy: User data (instructions, prompts) never leaves their machine for core functionality. The extension does not phone home or require authentication.

Cost: No server infrastructure to maintain. The extension is sustainable indefinitely without ongoing operational costs.

The trade-off is platform lock-in to Chrome (and Chromium-based browsers). Firefox and Safari support is technically feasible but deferred to focus development resources.

3.2 Technology Stack

The implementation uses a modern TypeScript-based stack optimized for extension development:

Language: TypeScript Provides type safety, IDE support, and refactoring confidence.

Build: esbuild Fast bundling with sub-second builds and native JSX transform.

UI Framework: React 18 Offers a proven component model, rich ecosystem, and developer familiarity.

State Management: Valtio Proxy-based reactivity with minimal boilerplate.

Fuzzy Search: Fuse.js Client-side fuzzy matching with configurable scoring.

Tree UI: react-arborist Hierarchical tree component with drag-drop support.

IndexedDB Wrapper: idb Promise-based IndexedDB access.

Valtio was chosen over Redux or Zustand for state management due to its proxy-based approach, which allows direct mutation of state objects while maintaining reactivity. This reduces boilerplate and makes state updates more intuitive in a tool where state changes frequently occur (typing, selection changes, panel toggles).

Fuse.js provides the fuzzy search capability for Quick Inclusion. Its configurable scoring system allows tuning to favor name matches over content matches, improving result relevance.

react-arborist provides the tree view component for the library pane. Its built-in support for drag-and-drop reordering eliminated significant custom implementation effort.

3.3 Chrome Extension Structure

3.3.1 Manifest V3 and components

PromptForge uses Chrome's Manifest V3 extension format, which represents the current standard for Chrome extensions. The manifest declares:

```
{
  "manifest_version": 3,
  "name": "PromptForge",
  "permissions": ["storage", "activeTab"],
  "host_permissions": [
    "https://chatgpt.com/*",
    "https://claude.ai/*",
    "https://gemini.google.com/*"
  ],
```

```
  "background": {
    "service_worker": "background.js"
  },
  "content_scripts": [{
    "matches": [
      "https://chatgpt.com/*",
      "https://claude.ai/*",
      "https://gemini.google.com/*"
    ],
    "js": ["content.js"]
  }]
}
```

The `host_permissions` grant access to the three supported webchat platforms. The content script automatically activates on these domains.

3.3.2 Sidebar injection

The sidebar is injected into webchat pages as an iframe, positioned along the left or right edge of the viewport. This approach provides:

CSS isolation: The iframe's separate document context prevents style conflicts between PromptForge and the webchat page.

Script isolation: JavaScript running in the iframe cannot directly access the webchat page's JavaScript context, preventing conflicts and providing security boundaries.

Consistent rendering: The sidebar renders identically regardless of the host page's styles or scripts.

The content script manages sidebar visibility, dimensions, and position. It also provides the bridge for webchat integration—reading from and writing to the webchat's input field.

3.3.3 Message passing

Chrome extensions use message passing for inter-component communication. PromptForge defines message types for:

- **Storage operations:** Read/write instructions, prompts, settings
- **Webchat integration:** Inject content, pull content, trigger send

- **UI coordination:** Sidebar open/close, focus management

The background service worker acts as the central hub, handling storage operations and routing messages between content script and sidebar.

3.4 Storage Architecture

3.4.1 Hybrid storage approach

PromptForge employs a hybrid storage strategy that balances persistence, capacity, and browser API constraints:

User-selected folder (File System Access API) stores instructions, lists, and prompts. This storage survives browser data clearing and has unlimited capacity constrained only by disk space.

IndexedDB stores the folder handle and UI state. This storage is ephemeral—cleared with browser data—but offers 50+ MB capacity.

chrome.storage.local stores settings. Like IndexedDB, it is ephemeral and offers up to 10 MB capacity.

This hybrid approach emerged from the limitations of each storage mechanism:

chrome.storage.sync has a 100KB limit—unusable for a library of instructions.

chrome.storage.local has a 10MB limit and does not survive "Clear browsing data"—risky for user content.

IndexedDB offers larger capacity but is also cleared with browsing data.

File System Access API provides true persistence to the user's filesystem, unlimited capacity, and survives browser reinstallation.

3.4.2 File System Access API for instructions

The File System Access API enables web applications to read and write files to a user-selected directory. PromptForge uses this API to store instructions, prompts, and custom lists as individual JSON files within a user-selected folder (e.g., `~/Documents/PromptForge/`). The folder contains three subdirectories: `instructions/` for instruction

files organized in a hierarchy mirroring the library pane, `lists/` for custom list files, and `prompts/` for prompt files.

Each instruction is stored as a separate JSON file. The folder hierarchy in `instructions/` mirrors the logical folder structure in the library pane.

3.4.3 Rationale for design choices

Per-file storage: Storing each instruction as a separate file enables clean git diffs for users who version-control their library. It also minimizes I/O—only modified files are rewritten on save.

User-selected folder: Letting users choose the storage location enables cloud sync (Dropbox, OneDrive, Google Drive), manual backup, and multi-machine usage without PromptForge needing to implement these features.

JSON format: Human-readable format allows manual inspection and editing outside the tool. JSON is well-supported across languages for potential interoperability.

IndexedDB for FileSystemDirectoryHandle: The File System Access API's directory handle must be stored in IndexedDB (not chrome.storage) because it requires structured cloning, which only IndexedDB supports.

3.5 Evolution of Architecture

The current architecture emerged through three distinct phases, each teaching lessons that informed the next.

3.5.1 Emacs origin

The project began as an Emacs package. Emacs Lisp provided rapid prototyping, and Org-mode offered a natural format for instruction storage. However, this approach had fundamental limitations:

- Emacs is a niche tool; most AI users do not use it
- No native webchat integration—required manual copy-paste

- Distribution challenges—Emacs package installation is non-trivial for non-Emacs users

The Emacs prototype validated the instruction composition concept but demonstrated the need for broader accessibility.

3.5.2 Webapp attempt

The second iteration was a web application with a Go backend. This provided:

- Cross-platform accessibility via any browser
- Server-side storage and sync
- Potential for collaboration features

However, the webapp approach introduced significant friction:

- Users had to switch between the webapp and their webchat
- Server infrastructure required ongoing maintenance and cost
- Privacy concerns—user data on external server
- Copy-paste remained necessary for webchat integration

The webapp demonstrated that minimizing context switches was essential for adoption.

3.5.3 Pure extension

The final architecture—a pure Chrome extension—eliminated the friction points of previous approaches:

- **No context switching:** Sidebar lives inside the webchat page
- **No server:** All functionality runs locally
- **Direct integration:** Content script can read/write webchat input field

- **One-click install:** Chrome Web Store distribution

The File System Access API, which became widely available in Chrome in 2020, made local-first storage viable without a native application.

This evolutionary path illustrates a common pattern in tool development: initial prototypes in familiar environments, followed by iterations that progressively reduce friction while preserving core functionality.

4 Data Model

This chapter specifies the data structures that underpin PromptForge: the instruction format, variable and embedding systems, prompts, and custom lists. These models balance expressiveness with simplicity, enabling sophisticated composition while remaining accessible to users who may inspect or edit files directly.

4.1 Instruction Model

4.1.1 JSON schema

Instructions are stored as individual JSON files with the following schema:

```
interface Instruction {
  id: string;           // UUID, immutable after creation
  name: string;         // Display name
  path: string;         // Path (e.g., "/analysis/deep")
  tags: string[];       // Categorization tags
  description: string;  // Human-readable description
  content: string;      // Instruction text
  created: string;      // ISO timestamp
  modified: string;     // ISO timestamp

  // Visibility flags (optional; default true)
  showInQuickInclusion?: boolean;
  includeInHarvest?: boolean;
  includeInRefine?: boolean;
  includeInExport?: boolean;
}
```

The `content` field holds the instruction text that will be inserted into prompts. This text may include variable placeholders (`\{\{variable\}\}`) and embed links (`[[id:uuid][name]]`) that are resolved at insertion time.

An example instruction file (`instructions/workflow/prompt-at-decisions.json`):

```
{
```

```
  "id": "f8af32d3-cadd-4ab2-8662-06231d54fd66",
  "name": "prompt-at-decisions",
  "path": "/workflow",
  "tags": ["interaction", "workflow"],
  "description": "Ask user to choose direction at ...",
  "content": "When encountering decision ...",
  "created": "2024-12-01T10:00:00Z",
  "modified": "2024-12-15T14:30:00Z"
}
```

4.1.2 UUID-based identification

Each instruction receives a UUID upon creation. This identifier is immutable—it never changes even if the instruction is renamed or moved to a different folder.

UUID-based identification provides referential stability. When one instruction embeds another via `[[id:uuid][name]]`, the reference survives renaming. If instructions were identified by name or path, renaming would break all references.

The trade-off is that UUIDs are not human-readable. Users cannot easily determine which instruction a UUID references without tool support. However, embed links include a display name (`[[id:uuid][Display Name]]`) that provides human context while the UUID provides machine stability.

4.1.3 Visibility flags

Four boolean flags control where an instruction appears (all default to `true`):

`showInQuickInclusion` Whether the instruction appears in Quick Inclusion search results.

`includeInHarvest` Whether the instruction is sent as library context in Harvest requests.

`includeInRefine` Whether the instruction is sent as library context in Refine requests.

`includeInExport` Whether the instruction is included when exporting the library.

These flags enable users to create instructions that serve specific purposes without cluttering unrelated contexts. For example, the bundled Refine instruction should not appear in Quick Inclusion (it is invoked via toolbar), so its `showInQuickInclusion` is false.

All instructions always appear in the library pane—there is no flag to hide them entirely. This design ensures users cannot lose access to their own content.

4.2 Variables and Embeds

4.2.1 Variable syntax

Variables enable parameterization of instructions. Rather than storing variable metadata separately, variables are defined inline in the instruction content using a templating syntax:

={{name}}= → Text input Free-form text entry.

={{name=default}}= → Text with placeholder Default value shown as placeholder.

={{name:a|b|c}}= → Dropdown Suggestions provided; user can still type custom values.

={{name=a:a|b|c}}= → Dropdown with default Suggestions with a preselected default.

={{name[]:a|b|c}}= → Multi-select chips Multiple values can be selected.

When an instruction with variables is previewed, the Preview Panel parses the content to extract variable definitions and displays input controls. On insertion, variables are replaced with their assigned values.

Example instruction content with variables:

```
Rank the following items by
{{criterium=importance:importance|complexity|urgency}}.
Consider {{aspects[]:cost|time|risk}} when making your
assessment.
```

This creates two inputs: a dropdown for `criterium` defaulting to "importance," and a multi-select for `aspects`.

4.2.2 Instruction embedding

Instructions can embed other instructions using wiki-link syntax inspired by Org-roam:

```
[[id:f8af32d3-cadd-4ab2-8662-06231d54fd66][prompt-at-decisions]]
```

When the containing instruction is inserted into a prompt, embeds are resolved recursively—the embedded instruction's content replaces the link.

Embedding enables composition. A complex instruction can be built from simpler components:

```
[[id:abc123][setup-context]]

{{task}}

[[id:def456][output-format]]
[[id:ghi789][quality-guidelines]]
```

When this instruction is inserted, each embed is replaced with the referenced instruction's content, producing a composite prompt.

Circular embed detection: The tool validates embed chains on save, rejecting any instruction that would create a circular reference (A embeds B which embeds A).

Broken references: If an embedded instruction is deleted, the link remains as text in the content. A warning indicator is planned but deferred to post-v1.

4.3 Prompts and Custom Lists

Prompts are the documents users compose in the editor. They are stored as JSON files in the `prompts/` directory:

```
interface Prompt {
  id: string;           // Sequential ID: "1", "2", "3"
  name: string;         // Display name (can be renamed)
```

```
  content: string;      // Prompt text
  created: string;      // ISO timestamp
  modified: string;     // ISO timestamp
}
```

New prompts receive sequential numeric IDs as default names. Users can rename prompts to meaningful titles while the ID remains stable.

Custom Lists provide cross-cutting organization that complements the folder hierarchy. A list contains references to instructions (and other lists) from anywhere in the library:

```
interface CustomList {
  id: string;           // UUID
  name: string;         // Display name
  path: string;         // Folder path for organizing lists
  description: string;  // Optional description
  instructionIds: string[];  // Referenced instructions UUIDs
  listIds: string[];    // UUIDs of other lists (for nesting)
  created: string;
  modified: string;
}
```

Lists store references, not copies. An instruction can appear in multiple lists while remaining in its original folder location. Changes to an instruction are reflected everywhere it appears.

This design mirrors the distinction between folders and tags/-playlists in other systems. Folders provide primary organization; lists enable secondary groupings that cut across the folder hierarchy.

5 User Interface

This chapter presents the PromptForge interface, focusing on the design decisions that enable efficient instruction discovery and insertion. Rather than exhaustively documenting every UI element, we emphasize the Quick Inclusion feature—the primary mechanism for rapid keyboard-driven interaction.

5.1 Layout Overview

The PromptForge interface is injected as a sidebar into supported webchat pages. Figure 5.1 shows the main interface layout.

The interface comprises four primary regions:

Header row: Contains the prompt selector dropdown, import/export buttons, and settings access. Users switch between prompts, create new prompts, and access global functions from this row.

Prompt Editor: The main text area where users compose prompts. Supports standard text editing plus instruction insertion via keyboard commands and drag-drop.

Library Pane: Displays the instruction library as a hierarchical tree. Users can browse folders, search by text or tags, and single-click to preview or double-click to insert. The pane is toggleable (Alt+L) and can be positioned on any edge.

Preview Panel: Shows instruction details when an instruction is selected. Includes the instruction body (editable), variable input controls, and insertion mode buttons. The panel appears below the editor and is toggleable (Alt+V).

The footer contains the webchat integration buttons: Pull (extract content from webchat), Inject (send prompt to webchat), and Inject & Send (inject and automatically submit).

5.2 Quick Inclusion

Quick Inclusion is the defining interaction pattern of PromptForge—a command palette that enables rapid, keyboard-driven instruction search and insertion without leaving the editor.

Figure 5.1: PromptForge main interface showing the prompt editor (left), library pane (right), and preview panel (bottom left)

5.2.1 Trigger and behavior

Quick Inclusion activates when the user types / in the editor after whitespace or at line start. A search panel appears below the editor, showing matching instructions with marginalia-style display: name, description excerpt, and content excerpt. Figure 5.2 shows Quick Inclusion in action.

As the user types, results filter in real time. Fuse.js provides fuzzy matching—=promdec= matches ”prompt-at-decisions” even without exact substring correspondence.

Typing // cancels Quick Inclusion and inserts a literal / character, allowing users to include slashes in their prompts without triggering the command palette.

5.2.2 Query syntax

Quick Inclusion supports query prefixes for targeted search:

Prefix	Effect	Example
(none)	Fuzzy match name, description, content	`/workflow`
`tag:` or `t:`	Filter by tag	`/t:api`
`folder:` or `f:`	Filter by folder path	`/f:analysis`

Multiple terms use implicit OR logic—an instruction matching any term appears in results. The library pane live-filters to show matching instructions while Quick Inclusion is open, providing visual feedback about what will be found.

5.2.3 Keyboard navigation

Quick Inclusion is designed for keyboard-only operation:

Key	Action
↓ / ↑	Navigate results
Tab	Autocomplete with selected name
Enter	Insert instruction using current mode
Alt+I/W/P/A	Insert with specific mode
Escape	Cancel, close panel

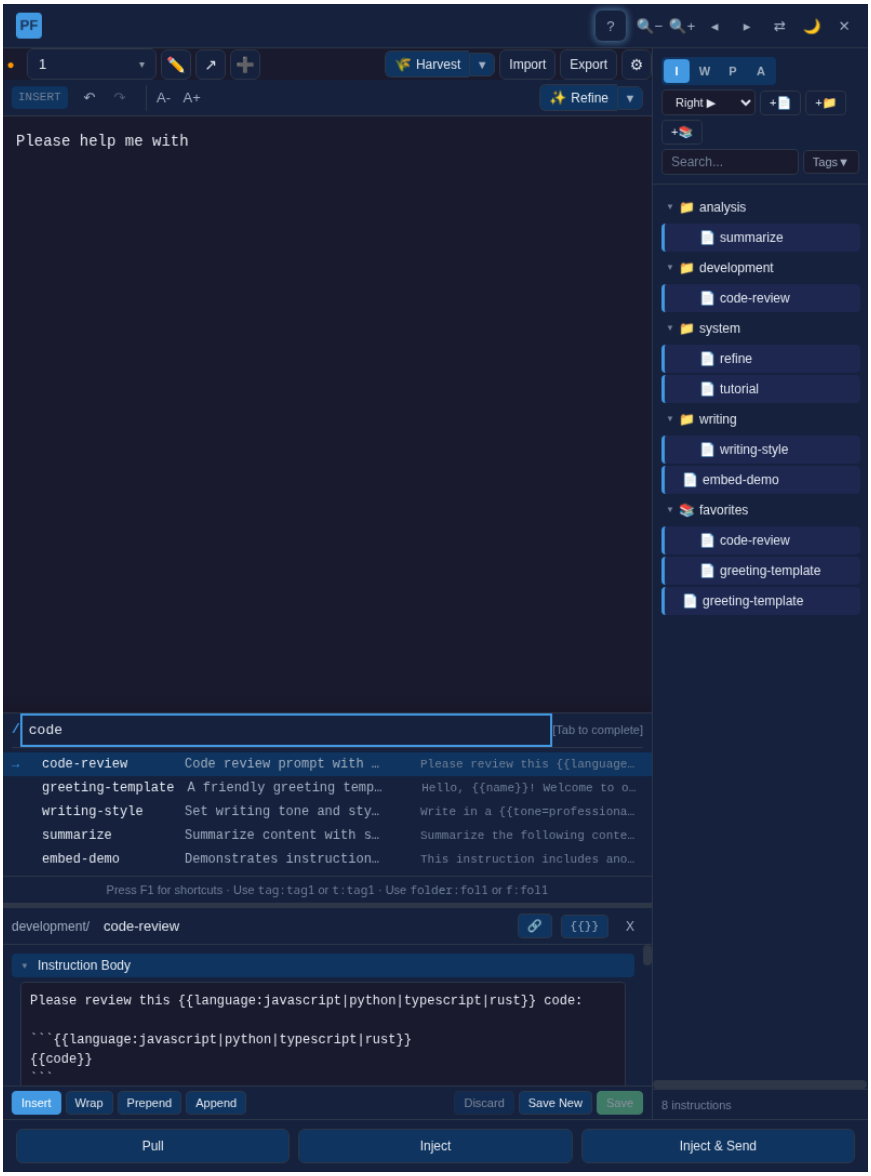

Figure 5.2: Quick Inclusion search panel showing fuzzy-matched results

The insertion mode (Insert, Wrap, Prepend, Append) follows a priority order: explicit Alt+key override > text selection (triggers Wrap) > current default mode. This allows rapid mode-specific insertion: `/ workflow` followed by Alt+P inserts at prompt start regardless of selected mode.

Quick Inclusion embodies the low-friction principle: experienced users can insert an instruction in under two seconds without touching the mouse.

6 Core Workflows

This chapter details the primary workflows that define PromptForge's functionality: webchat integration for seamless content transfer, insertion modes for flexible instruction placement, the Refine workflow for AI-assisted prompt improvement, and the Harvest workflow for instruction extraction. These workflows represent the operational core of the tool.

6.1 Webchat Integration

PromptForge's value proposition depends on tight integration with webchat AI interfaces. The tool must be able to send prompts to the webchat input field and retrieve content from it, all without requiring users to manually copy and paste.

6.1.1 Inject workflow

The Inject workflow transfers the current prompt from PromptForge's editor into the webchat's input field. Two variants exist:

Inject (Ctrl+Enter): Places the prompt content into the webchat input field without sending. This allows users to review or modify the prompt in the webchat before submission.

Inject & Send (Ctrl+Shift+Enter): Places the prompt into the webchat input and automatically clicks the Send button, submitting the prompt to the AI.

When the webchat input field already contains text, PromptForge displays a confirmation dialog asking whether to overwrite the existing content. This prevents accidental loss of partially-composed webchat messages.

Implementation requires site-specific DOM manipulation. The content script identifies the webchat's textarea or contenteditable element, sets its content via JavaScript (handling React's synthetic event system where necessary), and optionally triggers the send action. Each supported site requires tailored selectors and event sequences.

6.1.2 Pull workflow

The Pull workflow (Ctrl+Shift+L) extracts content from the webchat input field and creates a new prompt in PromptForge containing that content. This enables users to:

- Capture a prompt they started composing in the webchat
- Begin refinement of a prompt that was partially entered elsewhere
- Recover from situations where they accidentally composed in the webchat instead of PromptForge

Pull always creates a new prompt rather than inserting into the current prompt. This ensures users do not accidentally overwrite work in progress.

6.1.3 Supported sites

PromptForge integrates with three major webchat platforms:

Platform	Domain	Integration Status
ChatGPT	chatgpt.com	Full support
Claude	claude.ai	Full support
Gemini	gemini.google.com	Full support

Each platform presents distinct integration challenges:

ChatGPT uses a contenteditable div with React's synthetic event system. Direct DOM manipulation is insufficient; the content script must dispatch input events that React recognizes.

Claude uses a more standard textarea input, simplifying direct manipulation but requiring attention to focus management.

Gemini combines a textarea with additional UI chrome, requiring careful selector maintenance as Google frequently updates the interface.

The content script maintains separate integration modules for each platform, with site detection determining which module activates.

6.2 Insertion Modes

When a user inserts an instruction into a prompt, four modes control where and how the instruction text is placed. Understanding these modes is essential for effective prompt composition.

6.2.1 Insert mode

Insert mode places the instruction at the current cursor position, similar to typing. If text is selected, the instruction replaces the selection.

This is the default and most common mode, appropriate for most instruction insertions. The inserted text receives a brief visual highlight (green tint) to confirm placement, then the highlight fades.

6.2.2 Wrap mode and tagged parts

Wrap mode enables users to apply instructions to specific parts of their prompt. When text is selected and an instruction is inserted in Wrap mode:

1. The selected text is wrapped with numbered tags: `<1>...selected text...</1>`
2. A separator line is added at the end of the prompt (if not already present): `--- INSTRUCTIONS FOR TAGGED PARTS ---`
3. The instruction is appended after the separator with its corresponding number: `<1>: [instruction text]`

Example: A user writing about API design selects ”build a REST API for a bookstore” and wraps it with a ”deep-dive” instruction:

Before:

```
I need to build a REST API for a bookstore with
CRUD operations.
Consider pagination and filtering.
```

After wrapping ”build a REST API for a bookstore” with deep-dive instruction:

```
I need to <1>build a REST API for a bookstore</1> with
CRUD operations.
```

```
Consider pagination and filtering.

--- INSTRUCTIONS FOR TAGGED PARTS ---
<1>: Do a deep dive into this.
```

Multiple wraps accumulate with increasing numbers. The AI reads the tagged parts and their associated instructions, applying each instruction specifically to its marked content.

Wrap mode addresses a fundamental limitation of simple instruction prepending or appending: sometimes different parts of a prompt need different treatment. A user might want a deep analysis of one aspect but only a brief summary of another. Wrap mode makes this specificity expressible.

6.2.3 Prepend and Append modes

Prepend mode inserts the instruction at the very beginning of the prompt. This is appropriate for context-setting instructions that should frame the entire interaction: persona definitions, general behavioral guidelines, or meta-instructions like "Ask clarifying questions before proceeding."

Append mode inserts the instruction at the end of the prompt, but before any wrap separator if one exists. This is appropriate for output formatting instructions, quality requirements, or closing directives like "Summarize concisely."

Both modes preserve cursor position and scroll state—the user's view does not jump to the insertion point, minimizing disruption.

6.3 Refine Workflow

The Refine workflow enables AI-assisted prompt improvement. Rather than requiring users to manually think through potential enhancements, Refine asks the AI itself to suggest improvements based on the user's instruction library.

6.3.1 Purpose and rationale

Users often know their prompts could be better but lack the time or expertise to identify specific improvements. They may have accumulated

a library of useful instructions but forget to apply relevant ones. Refine addresses this gap by leveraging the AI as a prompt improvement assistant.

The key insight is that the AI already understands what makes effective prompts. By providing it with the user's instruction library and current prompt, we can ask it to suggest which instructions would improve the prompt and where they should be placed.

6.3.2 Process flow

The Refine workflow proceeds as follows:

1. **Initiation:** User clicks the Refine button or selects from the Refine dropdown to choose a specific refine instruction.
2. **Preview:** The Preview Panel displays the selected refine instruction, allowing the user to review or customize variables before proceeding.
3. **Request construction:** PromptForge constructs a request containing:
   - The refine meta-instruction (explaining the task and output format)
   - The user's current prompt (marked with `<PROMPT-TO-REFINE>` tags)
   - The user's instruction library as JSON (for reference)
4. **Injection:** The constructed request is injected into the webchat and sent to the AI.
5. **AI response:** The AI analyzes the prompt and library, responding with a marked-up version of the prompt containing suggested additions.
6. **Parsing:** PromptForge extracts the AI's response, parses the inline markup tags, and identifies individual suggestions.
7. **Review Mode:** The tool enters Review Mode (described below), allowing the user to accept or reject each suggestion.

6.3.3 Review mode

Review Mode provides fine-grained control over which suggestions to accept. When entered:

- The editor becomes read-only
- The prompt displays with numbered badges marking each suggestion
- A suggestions panel appears with checkboxes for each suggestion
- All suggestions are checked (accepted) by default

Figure 6.1 shows Review Mode with several suggestions in mixed states.

Users can:

- Toggle individual suggestions on/off
- Use Select All / Deselect All for bulk operations
- See immediate preview of how the prompt changes as suggestions are toggled
- Cancel to discard all suggestions and return to the original prompt
- Apply Selected to commit chosen suggestions

The working text is recomputed from scratch on each toggle, using the original prompt as the base. Suggestions are applied in position-descending order (right to left) to avoid offset shifting issues. This algorithm ensures that toggling suggestions in any order produces consistent results.

6.4 Harvest Workflow

The Harvest workflow extracts reusable instructions from successful conversations. Where Refine improves prompts using existing instructions, Harvest grows the instruction library by learning from what worked.

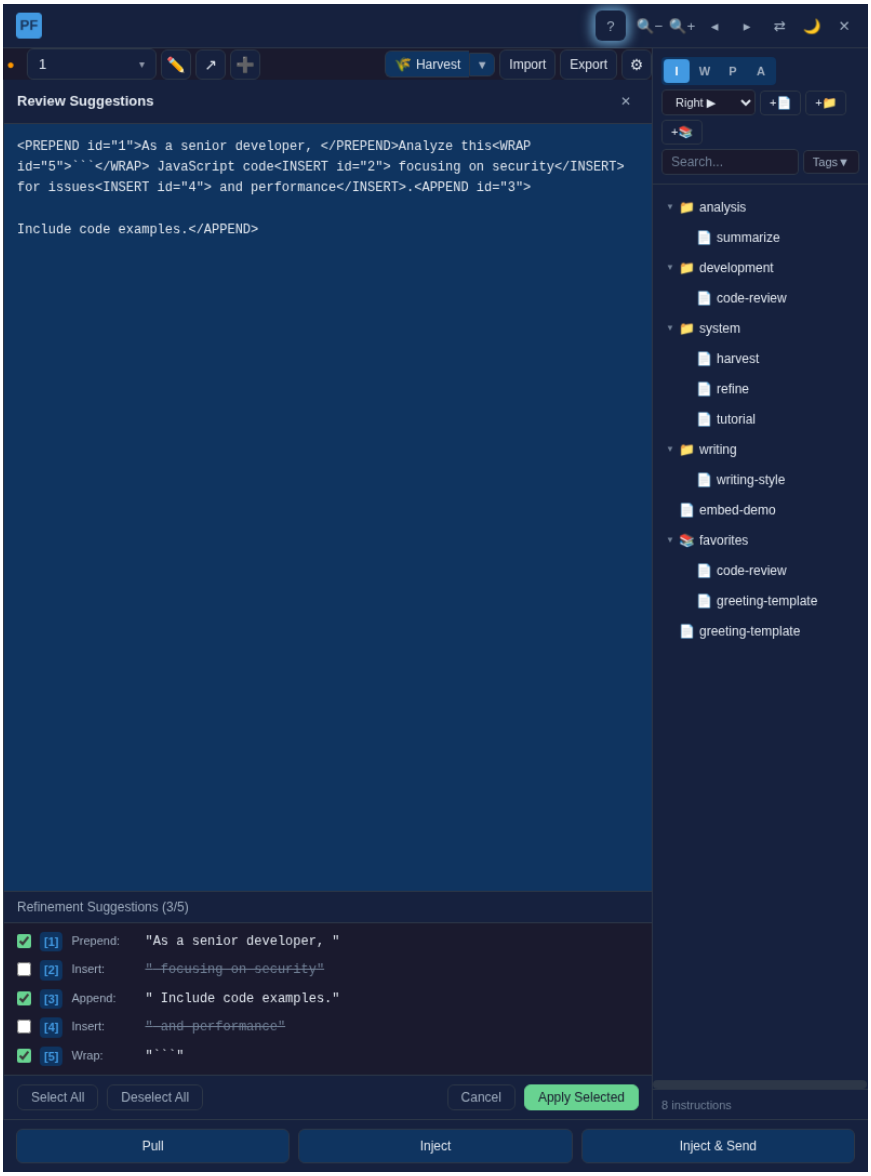

Figure 6.1: Review Mode showing suggestions with mixed accept/reject states

6.4.1 Purpose and rationale

Users naturally develop effective prompting patterns through conversation. They discover phrasings that elicit better responses, techniques that reduce misunderstandings, and structures that work well for particular tasks. Without explicit capture, these discoveries remain tacit knowledge—remembered vaguely or forgotten entirely.

Harvest automates the extraction process. After a productive conversation, users can ask the AI to identify reusable patterns from their messages and formulate them as instructions suitable for the library.

6.4.2 Process flow

The Harvest workflow proceeds similarly to Refine:

1. **Initiation:** User clicks the Harvest button after a productive conversation.
2. **Preview:** The Preview Panel shows the harvest meta-instruction, which can be customized.
3. **Request construction:** PromptForge constructs a request containing:
   - The harvest meta-instruction (explaining what to extract and output format)
   - The user's current instruction library as JSON (to avoid duplicates)
   - (The conversation context is already available to the AI)
4. **Injection:** The request is injected and sent.
5. **AI response:** The AI analyzes the conversation, identifying reusable patterns from user messages only (not system prompts or AI responses). It outputs a JSON array of suggested instructions.
6. **Import Modal:** PromptForge opens the Import modal populated with the suggested instructions, allowing the user to review, edit, and accept or reject each one.

The restriction to user messages is deliberate. Extracting from system prompts would contaminate the library with platform-specific content; extracting from AI responses could introduce hallucinated techniques. User messages represent genuine, proven prompting patterns.

6.5 Import and Export

Import and Export enable library portability: sharing instructions between users, backing up libraries, and migrating between machines.

Export produces a JSON array containing all exportable instructions (those with `includeInExport` true). The format matches the instruction schema, enabling manual inspection and editing:

```
[
  {
    "id": "abc123-def456",
    "name": "prompt-at-decisions",
    "path": "/workflow",
    "tags": ["workflow", "interaction"],
    "description": "Ask for direction at decision points",
    "content": "When encountering decision points...",
    "created": "2024-12-01T10:00:00Z",
    "modified": "2024-12-15T14:30:00Z"
  }
]
```

Import accepts JSON files or pasted JSON content. The Import modal parses the input and displays each instruction as an editable card, as shown in Figure 6.2. Instructions are classified by status:

NEW Does not exist in library; user can accept to add.

CHANGED Exists but differs; user can replace, keep, or import as new.

IDENTICAL Exact match; auto-skipped.

For CHANGED items, the modal displays an inline diff showing added text (green) and removed text (red). Users can choose to replace

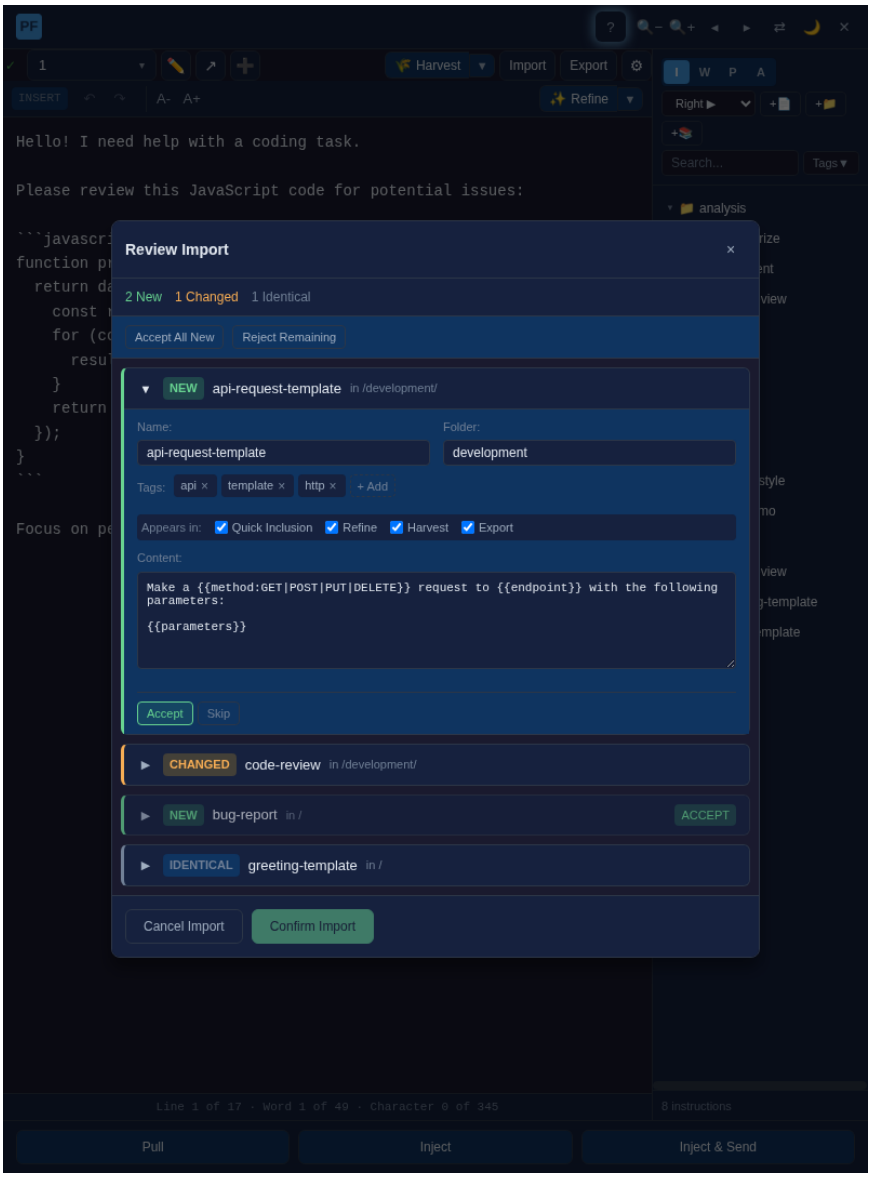

Figure 6.2: Import modal showing instructions with different statuses

the existing instruction, keep their current version, or import as a new instruction with a different name.

This conflict resolution mechanism ensures that importing never silently overwrites user work while still enabling library updates and merges.

7 AI Prompt Engineering

This chapter analyzes the AI instruction templates bundled with PromptForge: Refine, Harvest, and Tutorial. These templates demonstrate practical prompt engineering techniques and represent a significant design contribution of the tool. We examine their structure, the decisions that shaped them, and the prompt engineering principles they embody.

7.1 Design Principle

7.1.1 Leveraging webchat AI

A defining architectural decision of PromptForge is that AI-assisted features (Refine, Harvest, Tutorial) use the webchat AI rather than a separate backend. This approach has several implications:

No API configuration: Users do not need to obtain API keys or configure providers. The AI they are already conversing with handles the request.

Model flexibility: Users implicitly choose the AI model through their choice of webchat platform. Those who prefer Claude use Claude; those who prefer GPT-4 use GPT-4.

Conversation context: For Harvest, the AI already has access to the conversation history—no need to reconstruct or transmit it.

Transparency: Users see exactly what is sent to the AI and can read its response directly in the webchat.

The trade-off is that these features only work when integrated with a webchat (not in standalone mode), and the AI's response must be parsed from natural language rather than a structured API response.

7.2 Refine Instruction

The Refine instruction asks the AI to suggest improvements to a prompt by incorporating instructions from the user's library. This section analyzes its design.

7.2.1 Structure overview

The Refine instruction is structured in five sections:

1. **Task description:** Explains the role (suggest improvements to a prompt using available instructions)
2. **Input specification:** Defines the prompt-to-refine and instruction library, marked with XML-style tags for clear delineation
3. **Constraint definition:** Specifies that only additions are allowed—no modifications or deletions to existing text
4. **Output format specification:** Defines the inline markup tags for suggestions (`<refine-prepend>`, `<refine-append>`, `<refine-insert>`, `<refine-wrap>`)
5. **Detailed examples:** Provides worked examples for each suggestion type, demonstrating correct usage

7.2.2 Design decisions

Additions-only constraint: The instruction explicitly forbids the AI from modifying or deleting existing prompt text. This design decision ensures that the user's original intent is preserved; the AI can only enhance, not alter. This also simplifies the parsing and review process—every suggestion is an addition that can be cleanly accepted or rejected.

Inline markup format: Rather than asking for suggestions in a separate list, the instruction requires the AI to produce the refined prompt with markup tags inline. This makes the suggestions position-explicit: the AI must show exactly where each addition goes.

Numbered ID system: Each suggestion receives a unique numeric ID. Multiple tags can share an ID when they form a single logical suggestion (as with wrap mode, which requires opening tag, closing tag, and instruction reference). This enables atomic accept/reject at the suggestion level.

XML-style tags: The choice of `<refine-prepend id"N">=` syntax provides:

- Parseable structure
- Clear visual distinction from prompt content
- Attribute support for IDs
- Familiarity to developers

7.2.3 Prompt engineering techniques

The Refine instruction employs several documented prompt engineering techniques [1, 2]:

Role assignment: "You are presented with a prompt and a collection of instructions. Your task is to suggest improvements..."

Structured input demarcation: The `<PROMPT-TO-REFINE>` and `<INSTRUCTION-LIBRARY>` tags clearly separate the prompt from the library, preventing confusion about which is which.

Explicit constraints: The "Constraint: Additions Only" section uses imperative language to establish boundaries: "You may NOT modify or delete any existing text."

Format specification with examples: The instruction does not merely describe the output format—it provides worked examples for each tag type, showing correct usage in context.

Graduated complexity: Examples progress from simple (single prepend) to complex (all four modes combined), scaffolding the AI's understanding.

7.3 Harvest Instruction

The Harvest instruction asks the AI to extract reusable instructions from the conversation, formulating them in a format suitable for import into the library.

7.3.1 Structure overview

The Harvest instruction comprises:

1. **Task statement:** Analyze conversation, identify reusable patterns, output as instructions

2. **Source restriction:** Only extract from user messages, not system prompts or AI responses

3. **What to look for:** Positive guidance on extraction targets (techniques, meta-instructions, patterns)

4. **What NOT to extract:** Negative guidance excluding inappropriate content (context-specific, temporary, vague)

5. **Output format:** JSON schema matching the instruction model

6. **Context provision:** The user's current library (to avoid duplicates)

7.3.2 Design decisions

Source restriction: The explicit limitation to user messages prevents several failure modes:

- Extracting platform-specific system prompts
- Extracting AI-generated content that might be hallucinated
- Extracting templated responses not representative of user practice

Dual guidance (what to/what not to): Providing both positive and negative guidance reduces edge-case confusion. The AI knows to extract "meta-instructions that guide how the AI should behave" but not "corrections that were just fixing misunderstandings."

Schema matching: Requiring output in the exact instruction schema format simplifies import—no transformation needed.

Library provision: Including the current library helps the AI avoid suggesting instructions that duplicate existing ones.

7.3.3 Prompt engineering techniques

Enumerated criteria: Both "What to Look For" and "What NOT to Extract" use bulleted lists of specific criteria, reducing ambiguity.

Concrete examples in guidance: Parenthetical examples clarify abstract categories: "Meta-instructions that guide how the AI should behave (e.g., 'clarify before proceeding,' 'maintain a backlog')."

Empty-result handling: The instruction explicitly addresses the case where no instructions should be extracted: "If you don't find any worth extracting, respond with an empty array."

Format anchoring: The closing code block with `[]` provides a visual anchor for the expected JSON format.

7.4 Tutorial Instruction

The Tutorial instruction creates an interactive learning experience, teaching users how to use PromptForge through the webchat AI.

7.4.1 Structure overview

The Tutorial instruction is the most complex of the three, containing:

1. **Critical context:** Explanation of what the AI can and cannot observe (cannot see sidebar activity)
2. **Mode selection menu:** Four learning modes (Guided Tour, Quick Start, Reference, Specific Question)
3. **Mode-specific content:** Detailed scripts for each mode with phases, exercises, and response patterns
4. **Topic deep-dives:** Comprehensive reference content for each feature
5. **Graceful handling:** Patterns for managing user confusion, off-topic questions, and tutorial exit

7.4.2 Design decisions

Multi-modal structure: Different users have different learning preferences. The four modes accommodate:

- Guided Tour: Users who want structured introduction

- Quick Start: Users who want minimal information to start
- Reference Mode: Users who want self-directed exploration
- Specific Question: Users who already know what they need

Observability constraint acknowledgment: The instruction explicitly addresses a fundamental limitation: the AI cannot see what users do in the sidebar. This shapes the entire tutorial design—the AI must give clear instructions, wait for user confirmation or injected content, and trust verbal confirmations.

Response patterns: For each exercise, the instruction provides response templates keyed to different user behaviors (successful completion, confusion, skipping). This creates a more natural interactive experience.

Practice recognition: The instruction includes explicit guidance that when users inject practice prompts, the AI should acknowledge them as practice rather than attempting to answer them literally.

7.5 Common Patterns

Across the three instructions, several patterns recur:

Clear delineation of inputs: All three instructions use XML-style tags to mark input sections, preventing the AI from confusing instruction content with the meta-instruction itself.

Explicit output format specification: Each instruction defines its expected output format precisely, often with examples. This reduces format variability and simplifies parsing.

Constraint establishment: Each instruction explicitly states what the AI should not do, in addition to what it should do. Negative constraints often prove more important than positive guidance.

Graduated examples: Complex output formats are demonstrated through multiple examples of increasing complexity, scaffolding the AI's understanding.

Edge case handling: Instructions address edge cases explicitly (empty results for Harvest, confusion handling for Tutorial, multiple wrap tags for Refine).

Conversational framing: Each instruction establishes an appropriate register for the task—formal and precise for Refine and Harvest, friendly and pedagogical for Tutorial.

These patterns reflect accumulated experience in prompt engineering [4, 5] and represent transferable techniques applicable beyond PromptForge.

8 Development Journey

This chapter recounts the development history of PromptForge, tracing its evolution through three distinct architectural phases. Each phase taught lessons that informed the next, ultimately converging on the current Chrome extension design. We also discuss features that were considered but ultimately deferred, and reflect on the broader lessons learned.

8.1 Evolution of the Tool

8.1.1 Phase 1: Emacs-based prototype

PromptForge began as an Emacs package, developed over several months of continuous iteration. Emacs provided an ideal prototyping environment: Emacs Lisp enabled rapid experimentation, and the editor's extensibility allowed tight integration with existing workflows.

The prototype implemented rudimentary versions of many features that survive in the current tool: instruction composition, a searchable library, and variable substitution. A distinguishing feature was integration with Org-roam, a knowledge management system built on Org-mode. Instructions were stored as Org-roam nodes, enabling bidirectional linking and leveraging Org-roam's database for search and navigation.

Despite functional success within Emacs, this approach had a fundamental limitation: it was unsuitable for anyone who did not already use Emacs. Emacs has a steep learning curve and a small user base relative to the population of AI chat users. Requiring users to adopt an entire development environment to manage their prompts was an unacceptable barrier to entry. Even for technical users, the lack of webchat integration meant that prompts had to be manually copied between Emacs and the browser.

The Emacs phase validated the core concept—instruction-based prompt composition—while demonstrating that accessibility required a different platform.

8.1.2 Phase 2: Webapp attempt

The second iteration reimagined PromptForge as a web application with a Go backend. This architecture addressed the accessibility limitation: any user with a browser could access the tool, with no software installation required beyond navigating to a URL.

The Go backend performed well technically. It handled instruction storage, search, and synchronization reliably. The web frontend provided a modern, responsive interface accessible from any device.

However, two issues proved fatal to this approach:

Permissions complexity: The File System Access API, which allows web applications to read and write local files, requires explicit user permission grants. In the webapp context, these permissions had to be re-granted frequently, creating friction. More fundamentally, users were uncomfortable granting a web application access to their filesystem, even for a specific folder.

Manual server operation: For local deployment, users had to start the Go server manually and keep it running. This requirement, trivial for developers, proved unacceptable for the target user population. Users expected the tool to ”just work” when they opened their browser, not to require terminal commands and process management.

The webapp phase demonstrated that even a technically sound architecture can fail on user experience grounds. The friction of permissions and server management outweighed the benefits of the approach.

8.1.3 Phase 3: Pure Chrome extension

The final architecture emerged from the lessons of the previous phases. A Chrome extension could:

- Provide zero-friction installation via the Chrome Web Store
- Integrate directly with webchat pages, eliminating copy-paste
- Store data locally without the permissions friction of a webapp
- Run automatically without user intervention

The File System Access API, problematic in the webapp context, proved acceptable in an extension context. Users are accustomed to extensions requesting permissions, and the permission grant persists across browser sessions. The extension's service worker handles storage operations transparently.

This architecture sacrifices cross-browser compatibility (Chrome only for v1) in exchange for a dramatically improved user experience. The trade-off proved worthwhile: the tool is now usable by anyone who can install a Chrome extension.

8.2 Discarded Features

Several features were considered during development but ultimately deferred:

Firefox and Safari support: Cross-browser compatibility would expand the potential user base but requires maintaining separate extension codebases (Firefox uses different extension APIs, Safari requires additional tooling). This was deferred to focus development resources on completing the Chrome implementation.

Bulk list insertion: The ability to insert all instructions in a custom list with a single action was designed but not implemented. The interaction design—how to handle variables across multiple instructions, how to order insertions—required more thought than time permitted.

Instruction versioning: A version history for instructions would enable reverting changes and tracking evolution. This adds significant storage and UI complexity for a feature whose necessity is not yet proven by user demand.

These deferrals reflect a deliberate scope management strategy: ship a complete, polished core rather than a sprawling, unfinished feature set.

8.3 Lessons Learned

The development journey yielded several insights applicable beyond this project:

Iterate toward the right architecture: The path from Emacs to webapp to extension was not a failure of planning but a necessary exploration.

Each iteration revealed constraints and requirements that were not apparent in advance. The Emacs prototype proved the concept; the webapp attempt revealed that installation friction dominates technical elegance; the extension synthesized these learnings into a viable product.

User experience trumps technical sophistication: The Go backend was technically excellent—fast, reliable, well-structured. None of that mattered because users do not want to start a server every time. The lesson: evaluate architectures by the friction they impose on users, not by their elegance in isolation.

Browser APIs enable new possibilities: The File System Access API made local-first storage viable in a browser context. Manifest V3 extensions provide the security model and capability set needed for webchat integration. Staying current with platform capabilities enables architectures that would have been impossible years earlier.

Scope ruthlessly: The features deferred in Section 8.2 are all reasonable ideas. Implementing them would have delayed the core product indefinitely. Shipping a focused tool that does a few things well proved more valuable than a comprehensive tool that does many things poorly.

These lessons are not novel—they echo advice from software engineering literature. Their value lies in the concrete experience of learning them firsthand.

9 Future Work

This chapter identifies directions for future development of PromptForge, organized by scope and feasibility.

9.1 Near-Term Enhancements

Several enhancements are well-understood and could be implemented with moderate effort:

Cross-browser support: Porting to Firefox and Safari would expand the potential user base. Firefox's WebExtension API is largely compatible with Chrome's; Safari requires additional tooling but is increasingly viable. The main cost is ongoing maintenance of multiple codebases.

Bulk list operations: Inserting all instructions from a custom list with a single action would streamline workflows involving instruction sets. Design decisions around variable handling and insertion ordering need resolution.

Instruction versioning: A version history would enable reverting changes and tracking instruction evolution over time. This could be implemented as a lightweight log of diffs stored alongside each instruction file.

Theme customization: While light and dark themes exist, users cannot customize colors beyond this binary choice. A theme editor or CSS override mechanism would enable personalization.

9.2 Medium-Term Features

More ambitious features requiring significant design work:

Conversation-level refine: Analyzing an entire conversation to suggest improvements to the user's overall prompting approach, rather than a single prompt. This requires a more sophisticated refine instruction and UI for presenting conversation-wide suggestions.

Instruction analytics: Tracking which instructions are used most frequently, which produce the best results (via user feedback), and

which are never used. This data could inform library organization and suggest instructions to users.

Collaborative libraries: Sharing instruction libraries between users, with conflict resolution for collaborative editing. This would require either a server component or peer-to-peer synchronization.

9.3 Research Directions

Longer-term explorations that might inform future versions:

Automatic instruction generation: Rather than waiting for users to invoke Harvest, the tool could proactively suggest new instructions based on detected patterns in user prompts.

Instruction effectiveness measurement: Correlating instruction usage with conversation outcomes (user satisfaction, task completion) to empirically measure which instructions produce better results.

Natural language instruction search: Using LLM embeddings to enable semantic search across the instruction library, finding relevant instructions even when the query shares no words with the instruction text.

10 Conclusion

10.1 Summary of Contributions

This thesis presented PromptForge, a prompt engineering tool implemented as a Chrome browser extension. The primary contributions are:

1. **A data model for reusable prompt instructions** that supports composition through embedding, parameterization through variables, and cross-cutting organization through custom lists. The UUID-based identification system ensures referential stability across renames and moves.

2. **A pure browser extension architecture** that integrates seamlessly with ChatGPT, Claude, and Gemini webchat interfaces. The hybrid storage strategy—File System Access API for persistent user content, IndexedDB for handles and state, chrome.storage.local for settings—balances persistence, capacity, and user experience.

3. **AI-assisted workflows** that leverage the webchat AI itself rather than requiring separate API configuration. The Refine workflow enables AI-suggested prompt improvements with fine-grained accept/reject control via Review Mode. The Harvest workflow extracts reusable instructions from successful conversations.

4. **A keyboard-driven interface** centered on Quick Inclusion, providing FZF-style fuzzy search for rapid instruction discovery and insertion. Four insertion modes (Insert, Wrap, Prepend, Append) offer flexibility in instruction placement, with Wrap mode enabling targeted application of instructions to specific prompt sections.

5. **Documentation of the design journey** through three architectural phases (Emacs prototype, Go webapp, Chrome extension), illustrating how iterative development and user experience considerations shaped the final design.

10.2 Achievement of Thesis Goals

The thesis goals stated in Section 1.4 have been achieved:

Goal 1 (Data model design): Chapter 4 detailed the instruction model with its JSON schema, UUID identification, visibility flags, variable syntax, and embedding mechanism. The model supports the composition and parameterization requirements while remaining simple enough for manual inspection and editing.

Goal 2 (Browser extension architecture): Chapter 3 described the Manifest V3 extension structure, sidebar injection mechanism, message passing architecture, and hybrid storage strategy. The implementation integrates with three webchat platforms without server-side components.

Goal 3 (AI-assisted workflows): Chapter 6 explained the Refine and Harvest workflows, and Chapter 7 analyzed the prompt engineering techniques in the underlying instructions. These workflows demonstrate practical application of the ”leveraging webchat AI” design principle.

Goal 4 (Keyboard-driven interface): Chapter 5 presented Quick Inclusion and its query syntax. The interface supports both novice exploration (library browsing, preview panel) and expert efficiency (keyboard shortcuts, fuzzy search).

Goal 5 (Design documentation): Chapter 8 recounted the development journey, and design rationales are woven throughout Chapters 3–7. The thesis documents not just what was built, but why particular decisions were made.

10.3 Closing Remarks

PromptForge demonstrates that prompt engineering can be transformed from an ad-hoc practice into a systematic discipline through appropriate tooling. The instruction-based composition model provides a foundation for accumulating and applying prompt engineering knowledge. The browser integration eliminates friction that would otherwise discourage tool adoption. The AI-assisted workflows leverage the technology being used to improve the practice of using it.

The tool addresses a real need—many AI users develop effective prompting patterns but lack mechanisms to preserve and reapply them. PromptForge fills this gap while remaining accessible to non-technical users. Its local-first architecture respects user privacy and ensures that valuable instruction libraries remain under user control.

Prompt engineering as a field continues to evolve rapidly. New model capabilities, new prompting techniques, and new interaction patterns emerge regularly. PromptForge's extensible architecture—particularly the ability to customize AI instruction templates—positions it to adapt to this evolving landscape. The tool is not merely a product but a platform for prompt engineering practice.

References

1. SCHULHOFF, Sander et al. The Prompt Report: A Systematic Survey of Prompt Engineering Techniques. *arXiv preprint*. 2024. Available from arXiv: 2406.06608. Updated February 2025. Most comprehensive survey: 33 vocabulary terms, 58 LLM prompting techniques, 40 multimodal techniques. PRISMA review of 1,565 papers.
2. SAHOO, Pranab et al. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. *arXiv preprint*. 2024. Available from arXiv: 2402.07927. Updated March 2025. Categorizes 29 techniques by functionality. Includes taxonomy diagram and summary table.
3. CHEN, Banghao et al. Unleashing the Potential of Prompt Engineering for Large Language Models. *arXiv preprint*. 2023. Available from arXiv: 2310.14735. Updated May 2025. Covers chain-of-thought, self-consistency, generated knowledge. Addresses AI security and adversarial attacks on prompts.
4. WHITE, Jules et al. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. *arXiv preprint*. 2023. Available from arXiv: 2302.11382. Catalog of prompt patterns analogous to software design patterns. Reusable solutions to common problems.
5. The Prompt Canvas: A Literature-Based Practitioner Guide for Creating Effective Prompts. *arXiv preprint*. 2024. Available from arXiv: 2412.05127. Unified framework consolidating prompt engineering techniques. Discusses pre-designed templates and reusable prompts.
6. Wordflow: Social Prompt Engineering for Large Language Models. *arXiv preprint*. 2024. Available from arXiv: 2401.14447. Personal Prompt Library + Community Prompt Hub. Template system with prefix text and placeholder variables.
7. From Prompts to Templates: A Systematic Prompt Template Analysis for Real-world LLM Applications. *arXiv preprint*. 2025. Available from arXiv: 2504.02052. First systematic analysis of prompt templates in production LLM applications (Uber, Microsoft). Categorizes template components and placeholders.

8. Co-Writing with AI, on Human Terms: Aligning Research with User Demands Across the Writing Process. In: *Proceedings of the ACM on Human-Computer Interaction* (*PACMHCI*). ACM, 2025. Available also from: https://dl.acm.org/doi/10.1145/3757566. Systematic review of 109 HCI papers. Four design strategies: structured guidance, guided exploration, active co-writing, critical feedback.
9. Shaping Human-AI Collaboration: Varied Scaffolding Levels in Co-writing with Language Models. In: *Proceedings of the CHI Conference on Human Factors in Computing Systems* (*CHI 2024*). ACM, 2024. Available also from: https://dl.acm.org/doi/10.1145/3613904.3642134. Studies how varying AI input intensity influences user experience and writing outcomes.
10. Human-AI Collaboration Patterns in AI-assisted Academic Writing. *Studies in Higher Education*. 2024. Available also from: https://www.tandfonline.com/doi/full/10.1080/03075079.2024.2323593. 626 recorded activities analyzing doctoral student interactions with AI writing tools.
11. GOOGLE. *Chrome Extensions Documentation: Architecture Overview* [Chrome for Developers]. [N.d.]. Available also from: https://developer.chrome.com/docs/extensions/develop. Official documentation on extension architecture, Manifest V3, component communication.

11 Appendices

11.1 A. Keyboard Shortcuts

11.1.1 Global Shortcuts

Shortcut	Action
Ctrl+;	Open sidebar / Toggle focus
Ctrl+Shift+;	Open sidebar / Close sidebar
Ctrl+Enter	Inject prompt to webchat
Ctrl+Shift+Enter	Inject & Send (inject and submit)
Ctrl+Shift+L	Pull content from webchat
Ctrl+Z	Undo
Ctrl+Y	Redo
Alt+PageUp	Previous prompt in dropdown list
Alt+PageDown	Next prompt in dropdown list
Alt+L	Toggle library pane visibility
Alt+V	Toggle preview panel visibility
Escape	Cancel current operation / Close panel

11.1.2 Insertion Mode Shortcuts

Shortcut	Action
Alt+I	Set insertion mode to Insert
Alt+W	Set insertion mode to Wrap
Alt+P	Set insertion mode to Prepend
Alt+A	Set insertion mode to Append

11.1.3 Quick Inclusion Shortcuts

Shortcut	Action
//	Cancel and insert literal /
↑ / ↓	Navigate results
Tab	Autocomplete with selected instruction name
Enter	Insert selected instruction
Alt+I/W/P/A	Insert with specific mode
Escape	Close Quick Inclusion

11.1.4 Help

Shortcut	Action
?	Show help popup (when not in text input)
Ctrl+Shift+/	Show help popup

11.2 B. Full AI Instruction Templates

The following sections contain the complete text of the bundled AI instruction templates. These instructions are installed to the user's library on first run and can be customized. Additionally, they are present in the human-readable instruction collection (`instructions.pdf`) in the companion thesis.

11.2.1 Refine instruction

```
# Prompt Refinement Request

You are presented with a prompt and a collection of
instructions. Your task is to suggest improvements to the
prompt by incorporating suitable instructions from the
library.

## The User's Prompt

<PROMPT-TO-REFINE>
{prompt}
</PROMPT-TO-REFINE>

## The User's Instruction Library

<INSTRUCTION-LIBRARY>
{library_json}
</INSTRUCTION-LIBRARY>

## Constraint: Additions Only

You may only suggest additions to the prompt:
```

```
- **Prepend:** Add text at the very beginning
- **Append:** Add text at the end (but before the
  "--- INSTRUCTIONS FOR TAGGED PARTS ---" separator if
  present)
- **Insert:** Add text at a specific location
- **Wrap:** Surround a specific part with numbered tags and
  add instruction reference after separator

You may NOT modify or delete any existing text in the
user's original prompt.

## Inline Markup Format

Present your refined prompt using inline markup tags. Each
tag represents a suggestion that the user can individually
accept or reject.

**Tag Types:**
- `<refine-prepend id="N">...</refine-prepend>`
  - Add at the very beginning
- `<refine-append id="N">...</refine-append>`
  - Add at the very end
- `<refine-insert id="N">...</refine-insert>`
  - Add at a specific location
- `<refine-wrap id="N">...</refine-wrap>`
  - Part of a wrap operation

**Rules:**
- Every tag must have a numerical `id` attribute
- All tags share a single ID sequence (1, 2, 3, ...)
- Each unique id represents one suggestion
- Multiple tags with the same id are treated as a single
  suggestion
- The content inside the tag is new text

## Output Format

Put your entire refined prompt in a single code block with
```

```
your suggested additions using the described markup format.
```

11.2.2 Harvest instruction

```
# Instruction Harvesting Request

Analyze our conversation and identify any reusable
instructions, guidelines, or patterns that could be
extracted, generalized if needed, and saved to my
instruction library for use in future conversations with
LLMs.

## Source Restriction

Only use my messages (the user's messages) as the source
for potential new instructions. Do not extract instructions
from system prompts or your own messages.

## What to Look For

- Specific techniques, phrasings, or prompting methods that
  produced good results
- Meta-instructions that guide how the AI should behave
- Reasoning patterns (e.g., pro/con evaluation, ranking
  approaches)
- Output formatting conventions
- Interaction patterns (e.g., when to ask clarifying
  questions)
- Any concrete approach that could be generalized into a
  reusable instruction

## What NOT to Extract

- Content from system prompts or your responses
- Context-specific details that only apply to this
  conversation
- Temporary or one-time requests
- Subject-matter-specific questions I asked
```

```
- Corrections that were just fixing misunderstandings
- Instructions that are too vague to be reusable

## Output Format

Output your harvested instructions as a JSON array. Each
instruction should have: name, path, tags, description,
content. Do NOT include the id field.

## My Current Library

<CURRENT-LIBRARY>
{library_json}
</CURRENT-LIBRARY>

If you don't find any instructions worth extracting,
respond with an empty array: []
```

11.2.3 Tutorial instruction

The Tutorial instruction is extensive (~1000 lines) and provides an interactive learning experience with four modes: Guided Tour, Quick Start, Reference Mode, and Specific Question. Due to its length, only a summary is included here; the full instruction is available in the extension source code.

The instruction is structured as:

1. Critical context explaining the AI cannot observe sidebar activity
2. Mode selection menu presented to the user
3. Mode-specific content with phases, exercises, and response patterns
4. Comprehensive reference content for all features
5. Graceful handling patterns for user confusion and off-topic questions

The Tutorial demonstrates advanced prompt engineering techniques including multi-branch conversation flows, response templates keyed to user behavior, and explicit acknowledgment of system constraints.